

Alex Litchfield
Tomek Strzalkowski
Social Computing
19 March 2025

Homework 2 Report

1. The features I chose to derive from the textual content of the data were unigrams (individual words), bigrams (word pairs), and TF-IDF (Term Frequency-Inverse Document Frequency). The textual content derived from the data included the text content of each tweet and the description of the user. Unigrams were chosen as default to gloss over each word, and bigrams were added afterward as unigrams alone resulted in lower metrics. Certain words or word patterns could point to a prediction on the gender of the user. TF-IDF was used to weight the importance of certain words in the data, effectively nullifying common words such as “and”, “is”, or “the”.
2. Initially I was going to use either NB (Naïve Bayes) or SVM (Support Vector Machine) with a linear kernel. NB and SVM are both useful for text classification problems, but it can be said that SVM is better suited for larger datasets and more complex structures. NB, being a simple probabilistic classifier, requires little parameters to build its models and is often faster than other existing algorithms. It also assumes that a feature is independent of the others, which is useful when trying to compute “total likeliness” of something as opposed to a percentage chance overall. Since we only extracted “gender”, “text”, and “description” from each user, as well as enough preprocessing to cut down at least half the dataset, the dataset ended up being significantly less complex when provided to the model. This made NB better suited as this classifier is faster and more efficient in this scenario. Both classifiers are used in the code to show that NB makes better results, but the classifier that is chosen in the end is NB since it performed better.
3. K-fold cross validation involves the process of splitting the training set into k smaller sets, where during each iteration the current k set is used as the validation set and the remaining k-1 sets are used as the training set. I decided to use 5 fold cross validation, as anything below 5 results in worse end result metrics and anything higher did not significantly alter the metrics but increased the running time of the program. K fold cross validation was useful in this specific instance because it ensured that the model’s performance was not dependent on particular breakdowns of test/training data splits. It also reduced potential bias by training through multiple smaller sets. The metrics used to show the performance of the classifier were accuracy, precision, recall. The accuracy measured how many of the predictions were correct, the precision measured how many “female” labels provided by the model’s prediction were correct when compared with the true labels, and the recall measured how many “female” labels were actually correctly assigned when compared to the true labels.
4. Implementation Information
 - a. The data preprocessing was done in 2 stages. The first involved creating a dataframe that would be suitable for the second stage to modify. This dataframe was created through `pandas.read_csv()` for the homework 2 dataset. The homework 1 datasets each were parsed for names and text messages, which were then checked against the provided list of

names that correlated to each gender. These gender labels and text messages were then placed inside a pandas dataframe. The second stage would take the provided dataframe and remove common words like “and”, “is”, or “the” (stopword removal). The dataset would then have its text converted to numbers through TF-IDF (tokenization). Stage 1 utilized the pandas library to create dataframes. Stage 2 utilized `sklearn.feature_extraction.text` to get `TfidfVectorizer()`, used for preprocessing and feature extraction, and `sklearn.model_selection`, used to loop through k folds.

- b. Feature extraction was performed using `sklearn.feature_extraction.text` to get `TfidfVectorizer()`, which as stated before performed tokenization and stopwords removal. However, it also was used to limit vocabulary up to the 5000 most relevant words and apply unigrams and bigrams to determine patterns of word usage across the 2 genders.
 - c. My program uses both NB and SVM separately to compare which had the better results. For NB, I used `sklearn.naive_bayes` to get `MultinomialNB`, which trains on TF-IDF features in numerical format. For SVM, I used `sklearn.svm` to get `SVC(kernel = “linear”)`, which also trains on TF-IDF features. No normalization was applied to the TF-IDF features as the process of TF-IDF already applies normalization to the original data through altering the frequency of terms based on how often they are in the provided dataframe. This sentiment applies to both classifiers. Standardization was not applied since the TF-IDF values were in the range of 0 and 1.
 - d. The k fold cross validation utilized a $k = 5$ constant, which allowed the performance to remain consistent over time. When looping through each k fold, I first use `trainingSet = df.iloc[i]` and `testingSet = df.iloc[j]`, where i and j are the training and testing indexes of the current k fold, respectively. This selects the row to use from the dataframe for each set. Then, `trainTFIDF = vectorizer.fit_transform(trainingSet[“overallTextData”])` and `testTFIDF = vectorizer.transform(testingSet[“overallTextData”])` are used to apply feature extraction found in the vectorizer declaration to the datasets. Finally, the training set is applied to the classifier through `classifier.fit(trainTFIDF, trainingSet[“genderNums”])` and the prediction is created based off of the test data through `prediction = classifier.predict(testTFIDF)`. These will be used for the metrics in the current k fold. The libraries to obtain these functions were pandas and `TfidfVectorizer` through `sklearn.feature_extraction.text`.
 - e. The metrics for performance evaluation of the classifier were accuracy, precision, and recall, which were found using `accuracy_score(testingSet[“genderNums”], prediction) * 100`, `precision_score(testingSet[“genderNums”], prediction) * 100`, `recall_score(testingSet[“genderNums”], prediction) * 100` from `sklearn.metrics`. Each was multiplied by 100 at the end to be displayed as percentages later on. To get the average score for each metric, each metric’s score in a given k fold was appended to a list which then was summed and divided by its length, giving the mean. The output was printed in the jupyter notebook.
5. The performance of each classifier can be found in the jupyter notebook if each block is run, but by default the outputs have been cleared for the SVM classifier since it was not the chosen one

for this report. Charts are used in the jupyter notebook as well as standard printing of numbers, so only numbers will be shown in this report. As such, I will only be showing the metrics generated by the NB classifier in numerical format in this report.

HW2 Dataset (NB):

Accuracy for each of the 5 k folds respectively: 67.04%, 66.61%, 67.93%, 65.84%, 67.11%

Average accuracy across all k folds: 66.91%

Precision for each of the 5 k folds respectively: 66.87%, 64.89%, 66.78%, 65.28%, 66.25%

Average accuracy across all k folds: 66.01%

Accuracy for each of the 5 k folds respectively: 74.15%, 75.76%, 75.26%, 74.50%, 74.76%

Average accuracy across all k folds: 74.89%

6. Datasets A and B are kept separate since that's how they were provided originally in homework 1, and only numerical data will be displayed in this report as the charts are found within the jupyter notebook:

HW1 Dataset A (NB)

Accuracy for each of the 5 k folds respectively: 85.19%, 90.12%, 92.59%, 82.50%, 88.75%

Average accuracy across all k folds: 87.83%

Precision for each of the 5 k folds respectively: 85.19%, 90.12%, 92.59%, 82.50%, 88.75%

Average accuracy across all k folds: 87.83%

Accuracy for each of the 5 k folds respectively: 100.00% for each

Average accuracy across all k folds: 74.89%

HW1 Dataset B (NB)

Accuracy for each of the 5 k folds respectively: 59.35%, 66.39%, 68.03%, 66.39%, 65.57%

Average accuracy across all k folds: 65.15%

Precision for each of the 5 k folds respectively: 61.54%, 61.96%, 70.13%, 64.52%, 62.11%

Average accuracy across all k folds: 64.05%

Accuracy for each of the 5 k folds respectively: 78.87%, 90.48%, 77.14%, 88.24%, 90.77%

Average accuracy across all k folds: 85.10%

Analysis: It is worth noting that for each dataset, the 3rd k fold will typically have the highest accuracy and precision, while a tie can be seen between the 2nd and 5th k fold for highest recall (5th on HW1 dataset B and 2nd on HW2 dataset). This shows the need for the additional k folds, as without them, unseen data combinations would not have been used and would result in significantly higher average metrics across the board in all datasets.